

۱. هدف سند

این سند اصول و قوانین معماری Backend سیستم ERP SaaS را تعریف می‌کند تا تمامی توسعه‌های آینده بر اساس یک چارچوب مشخص، قابل توسعه، قابل نگهداری و استاندارد انجام شوند.

هدف اصلی:

- ایجاد Backend مقیاس‌پذیر برای یک سیستم ERP چندمستاجری (Multi Tenant)
- حفظ استقلال ماژول‌ها
- جلوگیری از وابستگی‌های مخرب بین بخش‌ها
- آماده‌سازی سیستم برای توسعه بلندمدت
- امکان تبدیل تدریجی به معماری Microservice در آینده در صورت نیاز

۲. تکنولوژی اصلی Backend

Framework

Laravel

Database

PostgreSQL

معماری ارتباطی

API First Architecture

Backend تنها مسئول ارائه سرویس‌های Business Logic از طریق API است و هیچ ارتباط مستقیمی بین Frontend و Database وجود ندارد.

۳. سبک معماری اصلی

معماری Backend بر اساس ترکیب اصول زیر طراحی می‌شود:

Modular Monolith

سیستم در ابتدا به صورت یک Backend واحد توسعه داده می‌شود، اما هر ماژول دارای مرزبندی داخلی مشخص، مستقل و قابل جداسازی خواهد بود.

هدف:

- کاهش پیچیدگی Microservice در ابتدای پروژه
- افزایش سرعت توسعه
- حفظ قابلیت جداسازی سرویس‌ها در آینده

Domain Driven Design (DDD)

منطق سیستم بر اساس دامنه‌های کسب‌وکار تقسیم می‌شود.

هر ماژول مسئول قوانین و رفتارهای مربوط به دامنه خود است و نباید منطق سایر دامنه‌ها را مدیریت کند.

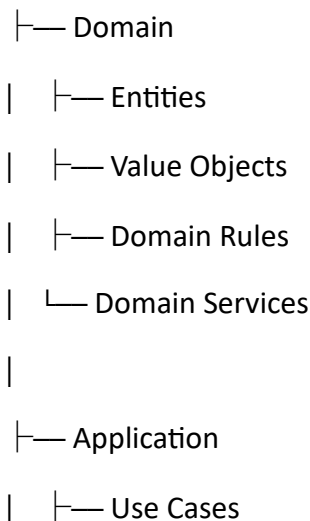
Clean Architecture

ساختار Backend باید از وابستگی مستقیم Business Logic به تکنولوژی جلوگیری کند.

۴. ساختار استاندارد هر ماژول Backend

هر ماژول باید دارای ساختار داخلی مشخص زیر باشد:

Module



| |— Application Services
| |— DTOs
|
|— Infrastructure
| |— Database
| |— Repositories
| |— External Services
|
|— API
| |— Controllers
| |— Requests
| |— Resources
|— Routes

۵. قانون استقلال لایه‌ها

قوانین زیر الزامی است:

- Domain نباید وابسته به Laravel باشد.
 - Domain نباید مستقیماً Database را صدا بزند.
 - Controller نباید شامل Business Logic باشد.
 - Repository مسئول ارتباط با Database است.
 - Service مسئول اجرای فرآیندهای کسب‌وکار است.
-

۶. قانون API First

تمام ارتباطات سیستم باید از طریق API انجام شود.

هیچ Frontend ، Mobile Application یا سرویس خارجی اجازه دسترسی مستقیم به Database ندارد.

API باید:

- Versioned باشد.
 - مستند باشد.
 - دارای استاندارد ثابت Request و Response باشد.
 - دارای Authentication و Authorization باشد.
-

۷. استاندارد API Versioning

تمام API ها باید نسخه داشته باشند.

فرمت استاندارد:

/api/v\/

هدف:

- جلوگیری از شکستن کلاینت های قدیمی
 - امکان توسعه نسخه های جدید
 - استقلال توسعه ماژول ها
-

۸. مدیریت دسترسی و امنیت

سیستم دسترسی باید فقط از Core Identity استفاده کند.

هیچ ماژولی اجازه ایجاد سیستم Authorization مستقل ندارد.

کنترل دسترسی باید بر اساس:

- User
- Role
- Permission
- Scope
- Tenant

انجام شود.

۹. قانون Multi Tenant

سیستم بر اساس Shared Database Multi Tenant طراحی می‌شود.

قوانین:

- تمام داده‌های عملیاتی باید متعلق به یک Tenant باشند.
- تمام Query ها باید Tenant Context داشته باشند.
- هیچ سرویس یا API اجازه خواندن یا نوشتن داده بدون بررسی Tenant را ندارد.
- Data Leakage بین Tenant ها خط قرمز سیستم است.

۱۰. قانون ارتباط بین ماژول‌ها

ارتباط مستقیم بین ماژول‌ها ممنوع است.

قوانین:

- ماژول‌ها نباید به جداول داخلی یکدیگر وابسته باشند.
- استفاده از Foreign Key بین ماژول‌های مستقل ممنوع است.
- ارتباط باید از طریق:

API ○

Domain Service ○

Event ○

Logical Reference ○

انجام شود.

۱۱. Database Access Pattern

دسترسی به Database باید از مسیر زیر باشد:

Controller



Application Service



Repository



Database

Controller نباید Query مستقیم اجرا کند.

۱۲. مدیریت Transaction

Transaction ها باید در سطح Application Service مدیریت شوند.

Repository نباید مسئول تصمیم‌گیری Business Transaction باشد.

۱۳. Event Driven آماده‌سازی آینده

ماژول‌ها باید به گونه‌ای طراحی شوند که امکان استفاده از Event وجود داشته باشد.

هدف:

- کاهش Coupling
- توسعه غیر همزمان
- آماده شدن برای Microservice

۱۴. مدیریت خطا

تمام API ها باید از استاندارد واحد Error Response استفاده کنند.

خطاها باید شامل:

- Error Code
- Message

- Validation Details

- Trace Identifier

باشند.

۱۵. Logging و Monitoring

تمام عملیات مهم باید قابل ثبت باشند.

موارد مهم:

- تغییرات داده
- ورود کاربران
- عملیات حساس
- خطاهای سیستمی

سیستم Logging نباید وابستگی مستقیم به Business Module ها ایجاد کند.

۱۶. Queue و پردازش‌های سنگین

کارهای سنگین نباید در Request اصلی اجرا شوند.

موارد مناسب برای Queue:

- ارسال پیام
- پردازش فابل
- گزارش‌گیری
- عملیات زمان‌بر

۱۷. Cache Strategy

استفاده از Cache باید کنترل شده باشد.

موارد مناسب:

- اطلاعات کم‌تغییر
- تنظیمات سیستم

- داده‌های پرتکرار
- Cache نباید منبع اصلی حقیقت داده باشد.

۱۸. آماده‌سازی برای **Microservice**

Backend فعلاً **Microservice** نیست.

اما قوانین زیر باید رعایت شود:

- مرز ماژول‌ها مشخص باشد.
- وابستگی‌ها کنترل شوند.
- Database Ownership مشخص باشد.
- ارتباطات قابل تبدیل به API باشند.

۱۹. استاندارد توسعه کد

قوانین:

- کد باید خوانا باشد.
- Naming Convention پروژه رعایت شود.
- از Duplicate Logic جلوگیری شود.
- تست‌پذیری در طراحی لحاظ شود.
- Business Rule‌ها مستند باشند.

۲۰. اصل نهایی معماری

هر تصمیم توسعه‌ای باید این سوال را پاسخ دهد:

"آیا این تصمیم باعث افزایش استقلال، توسعه‌پذیری و نگهداری آسان‌تر سیستم در آینده می‌شود؟"

اگر پاسخ منفی باشد، آن تصمیم نباید وارد معماری اصلی شود.

نتیجه نهایی

Backend سیستم **ERP SaaS** بر اساس:

- Laravel
- PostgreSQL
- API First
- Modular Monolith
- DDD
- Clean Architecture
- Multi Tenant Shared Database
- Event Ready Design

طراحی و توسعه خواهد شد.

این سند مرجع اصلی تمام تصمیمات معماری Backend پروژه است.